

1 Objetivo

Desenvolvimento de um programa em Matlab para fazer análise de imagens que representam envelopes com moradas que contêm elementos vários, em especial o texto do destinatário, incluindo um código postal em caracteres manuscritos da base de dados do MNIST (<http://yann.lecun.com/exdb/mnist>), que devem ser detetados e processados. O programa deve ser capaz de interpretar imagens fornecidas e de gerar os resultados pedidos e de os escrever num ficheiro, conforme descrito adiante. Serão dadas imagens exemplo para permitir o desenvolvimento, mas as imagens usadas para obter os resultados de avaliação serão novas. Embora uma parte significativa dos processos se possa fazer por técnicas tradicionais de visão artificial, para a deteção dos caracteres manuscritos devem usar-se técnicas usuais na Inteligência Artificial como redes neuronais, em particular as capacitadas para reconhecer os caracteres do *dataset* MNIST. Assim, além das técnicas de segmentação para isolar os elementos a classificar, será necessário definir e treinar uma rede neuronal que fará o reconhecimento o melhor possível da base de dados do MNIST e integrar a sua utilização no programa final.

2 Natureza das imagens

As imagens a processar terão vários elementos e alguns deverão ser detetados e classificados. A figura 1 ilustra um exemplo de imagem a processar onde se identificam de forma ilustrativa os potenciais elementos a detetar e/ou classificar, como descrito adiante.



Figura 1: Exemplo de uma imagem de envelope incluindo a **legenda dos elementos** a detetar e processar.

As características gerais das imagens são as seguintes:

- As imagens são a cores.
- As imagens podem estar rodadas e a zona do fundo fora do envelope será em nível de cinzento.
- As imagens contêm 4 linhas de texto em posição variável.
- As imagens podem conter um selo que poderá ter uma posição e orientação variável.
- Os envelopes podem ter cores diversas, como presente nos exemplos fornecidos.
- Nos envelopes pode haver outros elementos acessórios (símbolos, carimbos, etc.), como ilustrado nos exemplos disponibilizados, mas que não carecem de análise ou classificação para este trabalho.

3 Parâmetros a detetar em cada imagem

Em cada imagem, os parâmetros e propriedades a detetar pelo programa do aluno são:

- Tipo de selo dos possíveis da lista indicada na figura 2 (1,2,3, ...,7)
 - **Nota:** pode dar-se o caso de o selo estar ausente da imagem. Nesse caso o valor é 0 (zero).
- Número de palavras no nome do destinatário (2,3,...)
- Número de palavras na primeira linha da morada excluindo o número de porta (2,3,...)
- Número da porta à direita da primeira linha da morada (10 a 999)
- Dígitos do código postal (dois grupos de 4 dígitos cada um)



Figura 2: Ilustração dos selos possíveis de ocorrer nos envelopes e o seu número de identificação.

4 Características gerais dos elementos a detetar

Os objetos das imagens não apresentam variações de escala, mas poderão estar em posições de translação variáveis e eventualmente rodados em relação à horizontal. O selo poderá ter uma rotação adicional local para além da orientação geral do envelope.

Os caracteres do nome e morada do destinatário serão sempre no tipo de letra indicado nos exemplos (fonte e dimensão), exceto os do código postal que serão obtidos de forma aleatória do conjunto de caracteres da base de dados do MNIST.

Os envelopes podem ter fundos e bordos de cor variável e incluir elementos gráficos adicionais que não se sobrepõem aos elementos a detetar e a classificar, e que devem por isso ser descartados de processamento. A figura 3 ilustra alguns exemplos de imagens de envelopes a processar:



Figura 3: Exemplos de envelopes e dos seus conteúdos a analisar com um exemplo onde não há selo.

Os exemplos fornecidos *on-line* cobrem em média todos os limites de translação e rotação dos diversos elementos, e o mesmo se aplica à separação das palavras no nome e na primeira linha da morada. Recorda-se que o selo poderá não estar presente na imagem. Os fundos onde se encontram os envelopes são em nível de cinzento variável com a imagem.

5 Formato do ficheiro de resultados

Quando executado, o programa a desenvolver pelo aluno deve ler um conjunto de imagens fornecidas com nomes do género `svpi2026_TP2_img_MMM_NN.png` onde `NN` poderá variar de 01 até 99; esse número deverá ser detetado automaticamente pelo programa do aluno, como descrito adiante. `MMM` será um número de três dígitos que designa a sequência de imagens em análise e também deverá ser detetado automaticamente a partir no nome do ficheiro.

O programa a desenvolver pelo aluno deve analisar as imagens da sequência, uma por uma, e gerar uma tabela de estatísticas com tantas linhas quantas as imagens da sequência, e onde em cada linha se indicam os parâmetros pedidos. Esta tabela será sujeita a avaliação como descrito mais adiante, e deve ser escrita pelo programa do aluno num ficheiro com nome `tp2_nnnnnn.txt` onde `nnnnnn` é o número mecanográfico do aluno. Nesse ficheiro, as respostas para cada imagem devem aparecer numa linha e separadas por vírgulas. Por exemplo, para o aluno número 999999, o código Matlab do trabalho deve gerar o ficheiro `tp2_999999.txt` que deverá conter os seguintes valores separados por vírgulas:

`numMec` Número mecanográfico do aluno.
`numSeq` Número da sequência da imagem (Cf. nome do ficheiro de imagem).
`numImg` Número da imagem na sequência (Cf. nome do ficheiro de imagem).
`stampN` Número do selo presente no envelope (Cf. Figura 2), ou 0 se ausente.
`numNam` Número de palavras no nome do destinatário (2,3,...).
`numAdd` Número de palavras na primeira linha da morada excluindo o número da porta (2,3,...).
`numDoor` Número de porta que está à direita na primeira linha da morada (valor entre 10 e 999).
 `D1` Dígito 1 (mais à esquerda) do código postal (0 a 9)
 `D2` Dígito 2 do código postal (0 a 9)
 `D3` Dígito 3 do código postal (0 a 9)
 `D4` Dígito 4 do código postal (0 a 9)
 `D5` Dígito 5 do código postal (0 a 9)
 `D6` Dígito 6 do código postal (0 a 9)
 `D7` Dígito 7 do código postal (0 a 9)
 `D8` Dígito 8 do código postal (0 a 9)

6 Exemplo de resultados

Como exemplo concreto, tomem-se as duas imagens da figura 4 que se pretendem analisar e cujos ficheiros têm nomes: `svpi2026_TP2_img_120_35.png` e `svpi2026_TP2_img_120_50.png`.



Figura 4: Exemplos de duas imagens a processar: sequência 120, imagens 35 e 50.

Se o número mecanográfico do aluno fosse 999999, o ficheiro de resultados `tp2_999999.txt` para esta sequência de duas imagens teria o seguinte conteúdo:

```
999999,120,35,7,4,3,657,7,2,8,8,4,1,9,6
999999,120,50,2,3,2,41,3,9,0,8,8,9,5,6
```

Para mais fácil verificação, seguem os valores numa tabela com indicação do nome dos campos.

numMec	numSeq	numImg	stampN	numNam	numAdd	numDoor	D1	D2	D3	D4	D5	D6	D7	D8
999999	120	35	7	4	3	657	7	2	8	8	4	1	9	6
999999	120	50	2	3	2	41	3	9	0	8	8	9	5	6

7 Desenvolvimento do trabalho

O trabalho tem duas componentes que se complementam. Uma delas é o treino de uma rede neuronal para reconhecer os caracteres manuscritos do dataset do MNIST. Uma vez treinada a rede com resultados que o estudante considerar satisfatórios, deve-se guardar o modelo treinado num ficheiro com extensão `.mat` que deverá ter o nome `mnist_svpi2026_tp2_nnnnnn.mat` onde `nnnnnn` é o número mecanográfico do aluno. Se não for este o nome, não será possível avaliar o trabalho.

A outra componente, não menos importante, é o desenvolvimento de um programa para fazer todo o tratamento clássico da imagem, incluindo a separação dos dígitos manuscritos do MNIST que serão classificados pela rede neuronal já treinada e que este programa principal deve carregar quando executar através de um comando similar a:

```
load('mnist_svpi2026_tp2_nnnnnn.mat');
```

O programa principal deve ficar contido num único ficheiro Matlab do tipo `.m` e que terá o formato de função (*function*), devendo retornar um único valor que é o número mecanográfico do aluno. Se o aluno desenvolver funções auxiliares necessárias ao trabalho, elas deverão ficar incluídas nesse mesmo ficheiro, na parte final, depois da função principal.

Em resumo, as etapas sugeridas no desenvolvimento do trabalho são:

1. Fazer um programa para criar e treinar uma rede neuronal para aprender a classificar os caracteres manuscritos. Esse programa de treino da rede NÃO é para entregar para avaliação, é apenas de apoio.
2. Uma vez treinada a rede com o programa anterior, guardar a arquitetura da rede e seus pesos (modelo) num ficheiro designado `mnist_svpi2026_tp2_nnnnnn.mat` onde `nnnnnn` é o número mecanográfico.
3. Desenvolver o programa principal (designado `tp2_nnnnnn.m`) que fará todo o processamento de análise e segmentação da imagem e usará o modelo previamente treinado para classificar os caracteres manuscritos. Recordar-se que esses caracteres têm de estar num formato compatível com a rede neuronal, ou seja, tem de ter as mesmas dimensões das imagens dos caracteres de treino.
4. O programa principal deverá analisar as imagens e gerar o ficheiro de resultados `tp2_nnnnnn.txt`.

8 Procedimento para entrega do trabalho

O aluno deve executar o comando `SVPI_ProcessTP2_2026` (fornecido on-line) usando como argumento o nome do seu ficheiro Matlab. Por exemplo, se o programa do aluno se designar `tp2_999999.m` então, dentro da pasta que o contém, deve executar o comando:

```
SVPI_ProcessTP2_2026('tp2_999999.m')
```

Se o programa do aluno executar sem erros, será criado o ficheiro `cod_svpi2026_tp2_nnnnnn.vsz` (com `nnnnnn=999999` neste exemplo). Este ficheiro de arquivo conterá o programa do aluno (`.m`) e o ficheiro com o modelo treinado (`.mat`), e será o ficheiro que deve ser entregue no E-learning. O ficheiro com o modelo `mnist_svpi2026_tp2_nnnnnn.mat` deve forçosamente existir na mesma pasta, independentemente do seu conteúdo. Sem isso, o ficheiro `.vsz` para entrega no e-learning não será gerado e não será possível entregar o trabalho!

Adicionalmente, se o ficheiro de respostas `tp2_nnnnnn.txt` não for gerado pelo programa do aluno, então o comando `SVPI_ProcessTP2_2026` aborta em erro. Só o ficheiro com extensão `*.vsz` é aceite para entrega no E-learning e nenhum outro será considerado.

Resumo das condições necessárias para se conseguir a geração do ficheiro a entregar

`cod_svpi2026_tp2_nnnnnn.vsz`:

- O programa do aluno não pode ter erros de execução.
- A pasta onde está a ser executado não pode ter nenhum ficheiro `svpi2026_TP2_img_*.png`
- Os ficheiros de imagem `svpi2026_TP2_img_*.png` têm de estar na pasta anterior (`../`)
- O programa do aluno (formato de função Matlab) tem de devolver o número mecanográfico.
- O programa do aluno tem de gerar o ficheiro `tp2_nnnnnn.txt` onde `nnnnnn` é o número mecanográfico.
- O ficheiro com o modelo da rede neuronal `mnist_svpi2026_tp2_nnnnnn.mat` deve estar presente na mesma pasta do programa principal.

9 Avaliação

A avaliação é baseada em dois parâmetros principais:

1. Conformidade dos resultados do trabalho de acordo com imagens de avaliação.
2. Apreciação do código fornecido.

A conformidade dos resultados é feita essencialmente com base na taxa de resultados corretos em comparação com os resultados reais resultantes de uma análise correta às imagens de avaliação. A classificação dos dígitos do MNIST a considerar é a presente no *dataset* oficial. A apreciação do código fornecido poderá contemplar a eficiência geral, nomeadamente no tempo de execução. Programas com tempos de execução demasiado longos (mais de 30 segundos, em média, por imagem de uma sequência) poderão ser penalizados, ou mesmo excluídos de avaliação. O código entregue pelos alunos será sujeito a um sistema de verificação de plágio (sistema MOSS). Todos os trabalhos que apresentarem um índice de plágio maior do que um dado limite são passíveis de análise e tratamento específicos. Também o ficheiro `*.mat` será objeto de verificação e comparação.

10 Observações e recomendações sobre o trabalho e o Matlab

- O programa a executar deve-se chamar `tp2_nnnnnn.m` (onde `nnnnnn` é o número mecanográfico)
- O programa deve procurar as imagens na pasta anterior à sua (`../`).
- O programa deve correr de forma não interativa, ou seja, não deve ter interface gráfica nem deve esperar entrada de dados do utilizador.
- O programa deverá ser capaz de determinar quantas imagens a pasta tem, para as poder abrir.
- Para evitar eventuais erros prematuros de execução durante a avaliação, sugere-se que se usem construções do tipo `try - catch` nas partes mais sensíveis do código.

O programa deve gerar, na pasta corrente, o ficheiro `tp2_nnnnnn.txt`, (onde `nnnnnn` é o número mecanográfico do aluno) nos moldes descritos anteriormente.

As imagens a processar estarão na pasta anterior à pasta que contém o programa. Por exemplo, se o programa do aluno estiver numa pasta de nome parcial `"as_minhas_pastas_locais/svpi/tp2/"` então as imagens estarão em `"as_minhas_pastas_locais/svpi/"`.

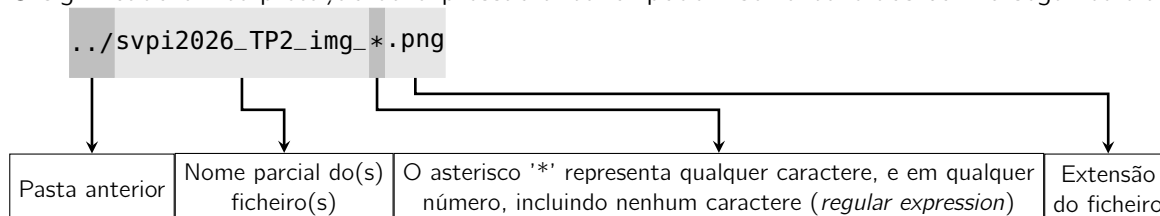
Ou seja, para tornar o programa independente do nome da pasta onde se encontre, ele deve procurar as imagens na pasta anterior. Assim, recomenda-se que essa pasta seja adicionada ao `path` do Matlab, e isso pode-se fazer recorrendo ao seguinte comando a colocar no início do programa:

```
addpath('../');
```

Para obter uma lista de ficheiros, para depois os poder processar, sugere-se o uso da função `dir()` do Matlab para listar o conteúdo de uma pasta. Neste caso, e para obter a lista de todas as imagens que estão na pasta anterior, pode-se fazer algo como:

```
listaF=dir('../svpi2026_TP2_img_*.png');
```

O significado e interpretação da expressão anterior podem ser entendidos com o seguinte diagrama:



Neste caso, a variável `listaF` conterá uma listagem de todos os ficheiros de imagens. A variável será uma estrutura em Matlab (*structure*) que conterá toda a informação sobre os ficheiros. Por exemplo:

```
size(listaF,1) -> dá o número de ficheiros encontrados pelo comando dir.
```

```
listaF(1).name -> dá o nome do primeiro ficheiro da lista.
```

Uma vez que a pasta anterior (onde estarão as imagens) já foi colocada no `path`, um comando como `A=imread(listaF(1).name)` não deverá ter problemas em encontrar o ficheiro.

11 Recurso a ferramentas de IA

É permitido a consulta e uso de ferramentas de IA no desenvolvimento do trabalho. Porém, se houver blocos de código gerado fundamentalmente por IA, cada um desses blocos deve estar numa função separada no fim do código principal escrito pelo aluno; se, por exemplo, uma certa função gerada por IA se chamar `extractAddress()`, então deverá ser renomeada como `LLM_extractAddress()` para identificar claramente que foi código gerado por uma LLM, qualquer que ela seja.

Embora não seja obrigatório, recomenda-se a inclusão em cada uma destas funções (sob a forma de comentário) a indicação de qual a LLM que gerou o código (Gemini, Claude, ChatGPT, etc.), bem como o *prompt* principal usado para pedir esse código. Não é garantido que estes elementos sejam levados em conta na classificação do trabalho, mas poderão ser úteis em situações de clarificação das soluções.